

2. Envoy

Envoy의 핵심 아키텍처는 무엇인가? Envoy는 **단일 프로세스 내에서 여러 워커 스레드를 활용하는 'embarrassingly parallel' 아키텍처**를 사용하여 높은 동시성과 낮은 지연 시간을 달성하며, 이는 컨테이너 환경에 최적화되어 있습니다.

1. Envoy의 목표와 아키텍처 개요

Envoy는 애플리케이션에게 네트워크를 투명하게 만들어, 개발자가 비즈니스 로직에 집중할 수 있도록 돕는 것을 목표로 한다.

1.1. Envoy 프로젝트의 시작 배경과 목표



1. 서비스 메시의 복잡성 해결

- 마이크로서비스 환경에서 개발자들이 네트워크 및 인프라 문제 디버깅에 많은 시간을 소비하는 문제를 해결하고자 한다.
- 개발자가 애플리케이션에 집중할 수 있도록 추상화된 네트워크를 제공하는 것이 목표이다.

2. Lyft의 마이크로서비스 도입 경험

- 2015년 Lyft는 마이크로서비스 도입 초기, 여러 언어(Python, PHP, Java 등)를 사용하며 복잡성으로 인해 도입이 중단될 위기에 처했다.

3. Out-of-process 프록시 설계

- a. 여러 언어를 지원하는 라이브러리 방식 대신, 언어에 독립적인 **out-of-process 프록시**로 설계되었다.
- b. HAProxy, Nginx와 같은 기존 프록시들과 경쟁하기 위해 **저지연, 고성능**을 목표로 했다.

4. 주요 설계 목표

- a. **L3/L4 필터 아키텍처**: 다양한 프로토콜(Redis, HTTP, MongoDB, Kafka 등)을 지원하는 **바이너리 프록시**로 설계되었다.
- b. **HTTP/2 지원**: 당시 대부분의 프록시가 지원하지 않았던 **HTTP/2 풀 프록싱**을 지원한다.
- c. **서비스 구성 및 검색**: 운영 경험을 바탕으로 **API를 통한 동적 구성 관리** 기능을 중요하게 고려했다.
- d. **능동/수동 상태 확인**: 능동 및 수동 상태 확인 기능을 제공한다.
- e. **고급 로드 밸런싱**: 로드 밸런싱, 속도 제한, 서킷 브레이킹, 타임아웃 등을 지원한다.
- f. **관찰 가능성(Observability)**: 최고 수준의 통계(stats), 로깅, 추적(tracing)을 통해 복잡한 마이크로서비스 환경에서의 문제 해결을 돕는다.
- g. **운영 민첩성**: 엣지 프록시와 내부 서비스 프록시 간에 **동일한 코드베이스**를 사용하여 운영 효율성을 높인다.

1.2. Envoy의 고수준 아키텍처

Envoy design goals

- Out of process architecture
- Low latency, high performance, dev productivity
- L3/L4 filter architecture
- HTTP L7 filter architecture
- HTTP/2 first
- Service/config discovery
- Active/passive health checking
- Advanced load balancing
- Best in class observability
- Service/middle/edge proxy
- Hot restart

Videos Sponsored by Google Cloud

1. 연결 처리 파이프라인

- 리스너 필터(Listener Filters):** 연결이 처리되기 전에 작동하며, 연결 자체에 대한 정보를 조작한다.
- TCP/L4 필터(TCP/L4 Filters):** 연결이 설정된 후 작동하며, Redis, MongoDB, HTTP 등 프로토콜별 처리를 담당한다.
 - 여러 필터를 체인 형태로 구성하여 인증, 속도 제한, 프로토콜 스니핑 등의 기능을 조합할 수 있다.
- HTTP 연결 관리자(HTTP Connection Manager):** L3/L4 필터의 일종으로, 바이트를 파싱하여 헤더, 본문, 트레일러와 같은 HTTP 메시지를 생성한다.
- HTTP 계층 필터(HTTP Layer Filters):** 헤더, 본문, 트레일러를 기반으로 작동하며, 인증, 속도 제한, 버퍼링 등을 HTTP 메시지 수준에서 처리한다.
- 라우터 필터(Router Filter):** 서비스 라우팅을 담당하는 필터이다.

2. 클러스터 관리자(Cluster Manager): Envoy가 라우팅할 수 있는 백엔드 서버 그룹(클러스터)을 관리한다.

- 클러스터(Cluster):** 호스트(백엔드 서버)들의 그룹이다.

3. 중앙 집중식 기능

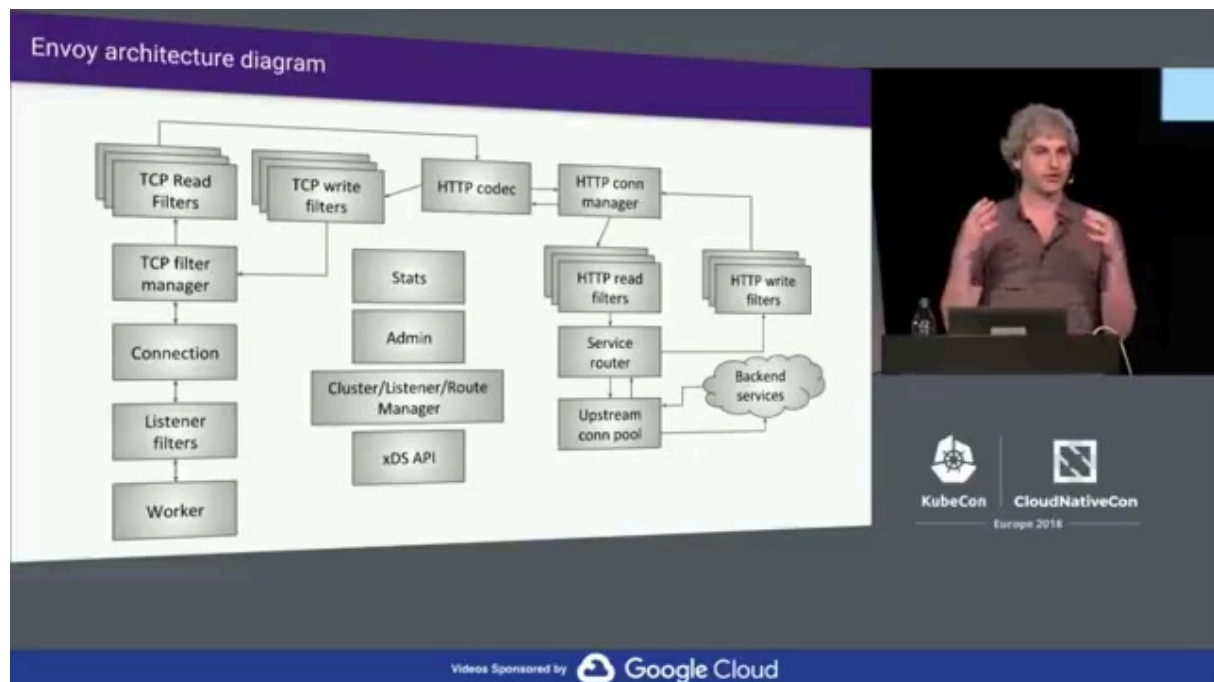
- 통계(Stats), 관리(Admin):** 풍부한 정보를 제공하여 디버깅에 유용하다.
- 리스너 관리자, 라우트 관리자:** 수신 트래픽을 처리하고 필터 스택을 설정한다.

- c. **xDS (Discovery Service API)**: 리스너, 클러스터, 엔드포인트 등의 정보를 동적으로 받아 관리자들에게 전달한다.

2. Envoy의 스레딩 모델과 동시성 처리

Envoy는 단일 프로세스 내에서 메인 스레드와 다수의 워커 스레드를 활용하는 'embarrassingly parallel' 아키텍처를 통해 높은 동시성과 낮은 지연 시간을 달성한다.

2.1. 동시성 처리 방식의 진화



1. 과거의 스레드 모델 (Connection per Thread)

- a. 운영체제 스레드당 하나의 연결을 처리하는 방식은 메모리 부족과 잦은 컨텍스트 스위칭으로 인해 확장성이 낮다.
- b. 10,000개 이상의 연결을 처리하는 C10k 문제 해결에 부적합하다.

2. 비동기 이벤트 루프 모델 (Async Event Loop)

- a. Nginx에서 시작된 방식으로, 하나의 스레드가 여러 연결을 비동기적으로 처리한다.
- b. 프로그래밍이 복잡해지지만, 높은 연결 수를 처리하기 위한 필수적인 방식이다.
- c. 코루틴과 같은 새로운 방식도 연구되고 있다.

2.2. Envoy의 스레딩 모델

Envoy threading model (c10k)

- Connection per thread does not scale
- Scaling requires many connections per thread: "c10k"
- Requires *asynchronous* programming paradigms: harder

KubeCon CloudNativeCon Europe 2018

Videos Sponsored by Google Cloud

1. 단일 프로세스, 다중 스레드 구조

- Envoy는 여러 프로세스가 아닌 **단일 프로세스** 내에서 스레드를 활용한다.
- 프로세스 간 통신보다 스레드 간 통신이 더 간단하고 운영 관리가 용이하기 때문이다.

2. 메인 스레드(Main Thread)

- 저처리량, 고중요도** 작업을 담당한다.
 - xDS 호출, 런타임 설정 로드, 통계 플러시, 관리 서버, 시그널 처리 등이 포함된다.
- CPU 집약적이지 않으며, **락(Lock)**이 필요 없어 추론하기 쉽다.
- Envoy에는 **스레드 풀**이 존재하지 않는다.

3. 워커 스레드(Worker Threads)

- 하드웨어 스레드 수에 맞춰** 기본적으로 1:1로 생성된다. (예: 4코어 CPU → 4개 워커 스레드)
- 데이터 플레인**을 담당하며, 리스너를 호스팅하고 연결을 수락하며 필터 파이프라인을 처리한다.
- 'Embarrassingly Parallel'** 아키텍처를 구현한다.
 - 워커 스레드 간 **별도의 조정(Coordination)**이나 락이 거의 필요 없어 대규모 코어 수에 확장 가능하다.

- 이는 90개 이상의 코어를 가진 최신 서버 환경에 최적화된 설계이다.
- d. **파일 I/O**와 같이 특정 작업은 블로킹될 수 있어, 이러한 경우 별도의 스레드로 기능을 분리한다.

2.3. RCU (Read-Copy Update)와 TLS (Thread-Local Storage)

Envoy threading model (overview)

- Main thread handles non-data plane misc tasks
- Worker thread(s) **embarrassingly parallel** and handle listeners, connections, and proxying
- File flush threads avoid blocking
- Designed to be **100% non-blocking**
- Designed to scale to **massive parallelism** (# of HW threads)

Diagram components:

- Main thread:** xDS, Runtime, Stat flush, Admin, Process management
- Worker thread(s):** Listeners, Connections
- File flush thread(s):**

KubeCon CloudNativeCon Europe 2018

Video Sponsored by Google Cloud

1. RCU (Read-Copy Update)

- a. **읽기-복사-업데이트** 동기화 패러다임으로, 높은 동시성과 읽기 위주의 워크로드에 적합하다.
- b. **읽기 시 락을 잡지 않아** 여러 스레드에서 데이터를 동시에 읽을 수 있다.
- c. 업데이트 시, 원본 데이터를 복사하여 새로운 데이터를 생성하고, '**무정지 시간 (Quiescent Period)**' 동안 기존 데이터를 사용하던 스레드들이 모두 종료된 후에 새로운 데이터로 교체한다.
 - C++에서는 **shared_ptr**을 사용하여 구현된다.
- d. 락 없이 데이터를 업데이트할 수 있어 성능 향상에 기여한다.

2. TLS (Thread-Local Storage)

- a. 각 스레드마다 독립적인 데이터를 저장하는 메커니즘이다.
- b. Envoy에서는 RCU 데이터와 결합하여 사용된다.

- c. **TLS 슬롯(TLS Slots)**을 할당받아 사용하며, 이는 포인터의 벡터(vector) 형태로 관리된다.
- d. 메인 스레드에서 RCU를 통해 업데이트된 데이터가 각 워커 스레드의 TLS 슬롯에 복사되어 **결과적으로 일관성(eventually consistent)**을 유지한다.
 - 이를 통해 락 없이 데이터를 안전하게 업데이트하고 사용할 수 있다.

2.4. TLS와 RCU를 활용한 클러스터 업데이트 예시

Envoy threading model (TLS and RCU)

- TLS = Thread Local Storage
- TLS slots can be allocated dynamically by objects
- RCU is used to post shared read-only data from the main thread to workers

Event loop post()

Main 0 1 2 3 4 5 "Slots"

Worker 1 0 1 2 3 4 5

Worker 2 0 1 2 3 4 5

Worker 3 0 1 2 3 4 5

Worker 4 0 1 2 3 4 5

TLS get()

KubeCon CloudNativeCon Europe 2018

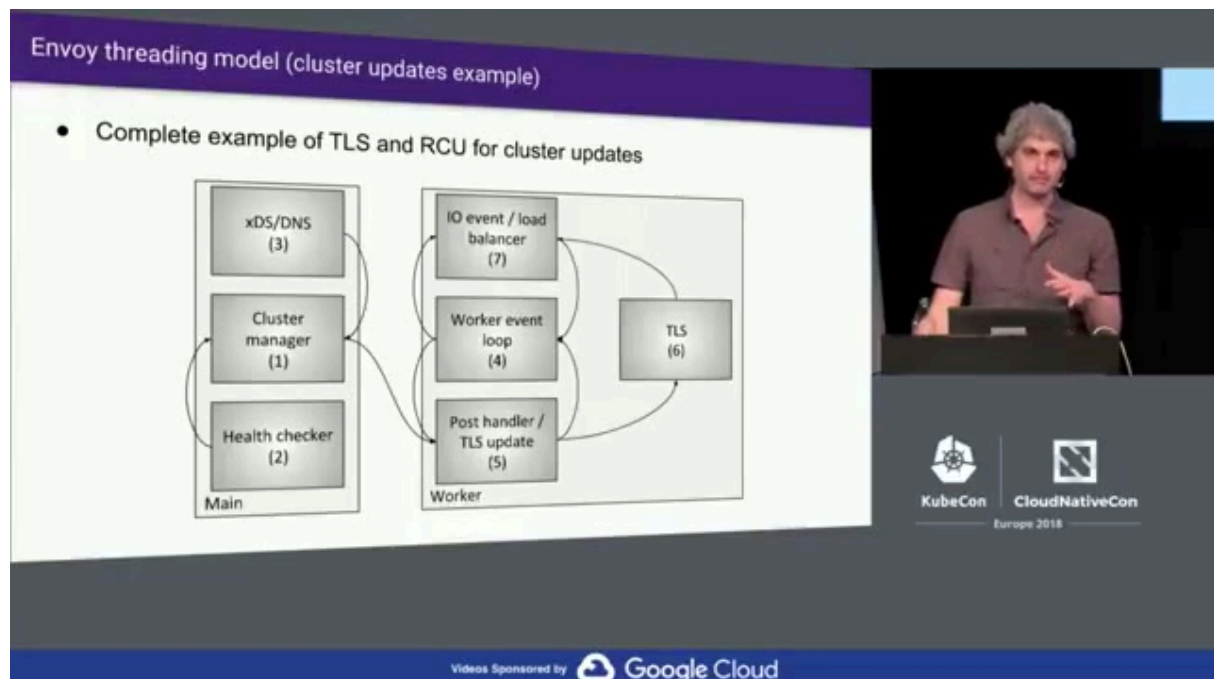
Videos Sponsored by Google Cloud

1. 클러스터 관리자는 다양한 소스(헬스 체크, 서비스 검색 등)로부터 정보를 받아 클러스터(백엔드 서버 그룹) 상태를 관리한다.
2. 호스트 상태 변경 시, 클러스터 관리자는 메인 스레드에서 새로운 호스트 목록을 계산하고 **참조 카운트된(reference-counted) 배열**로 만든다.
3. 이 새로운 호스트 목록은 **TLS 슬롯을 통해 RCU 방식으로 모든 워커 스레드에 전파**된다.
4. 워커 스레드는 이벤트 루프 내에서 해당 TLS 슬롯의 데이터를 업데이트하고, 로드 밸런싱 등 의사 결정 시 이 데이터를 **락 없이** 사용한다.
5. 이 방식은 '**embarrassingly parallel**' 하여 무한대에 가까운 스레드 수로 확장 가능하다.

3. 핫 리스타트와 통계 처리

핫 리스타트는 Envoy가 연결을 끊지 않고 바이너리 또는 설정 업데이트를 수행할 수 있게 하여, 서비스 중단을 최소화한다.

3.1. 핫 리스타트(Hot Restart)



1. **목표:** Envoy 바이너리 또는 설정 업데이트 시 **연결을 끊지 않고 재시작**하는 기능이다.
2. **필요성:** 컨테이너 환경이 아닌 가상 머신이나 베어메탈 환경에서 블루/그린 배포가 어려운 경우, 서비스 중단 없이 업데이트를 수행하기 위해 필수적이다.
3. **동작 방식**
 - a. **두 개의 Envoy 프로세스 동시 실행:** 새로운 Envoy 프로세스가 시작되고, 기존 프로세스와 통신한다.
 - b. **공유 메모리 영역(Shared Memory Region):** 주로 통계(stats) 데이터를 공유하며, 카운터, 게이지, 히스토그램의 일관성을 유지한다.
 - 특히 게이지(예: 활성 연결 수)의 일관성은 모니터링에 중요하다.
 - c. **경량 RPC 프로토콜 (UNIX Domain Sockets):** 두 Envoy 프로세스 간 버전 호환성 확인 및 소켓 전달(socket passing)을 수행한다.
 - d. **소켓 전달(Socket Passing):** Linux의 소켓 전달 기능을 사용하여 기존 연결을 새로운 프로세스로 넘겨준다.
 - e. **점진적 전환:** 새로운 Envoy가 트래픽을 받기 시작하면, 기존 Envoy에게 연결 드레인(drain)을 시작하도록 알린다.

- f. **안정적인 종료:** 설정된 드레인 시간(15분, 1시간 등) 후 기존 Envoy가 종료된다.

4. 컨테이너 환경에서의 장점

- a. 프로세스 래퍼 없이 통신이 가능하여 컨테이너 환경에서도 핫 리스타트가 가능하다.

3.2. 통계(Stats) 처리

Envoy hot restart (mechanics)

- Stats/locks kept in shared memory
- Simple RPC protocol over unix domain sockets (UDS)
- Sockets, some stats, etc. passed between processes
- Built for containers

KubeCon CloudNativeCon Europe 2018

Videos Sponsored by Google Cloud

1. 구성 요소

- a. **스토어(Store):** 카운터, 게이지, 히스토그램 데이터를 저장한다.
- b. **싱크(Sink):** Prometheus, StatsD, gRPC 등 다양한 통계 백엔드로 데이터를 전송하는 프로토콜 어댑터이다.
- c. **관리 서버(Admin Server):** 통계를 풀(pull) 방식으로 제공하는 엔드포인트이다.
- d. **스코프(Scope):** 통계 그룹으로, 함께 삭제될 수 있어 메모리 누수를 방지한다.

2. 고성능 통계 처리 메커니즘

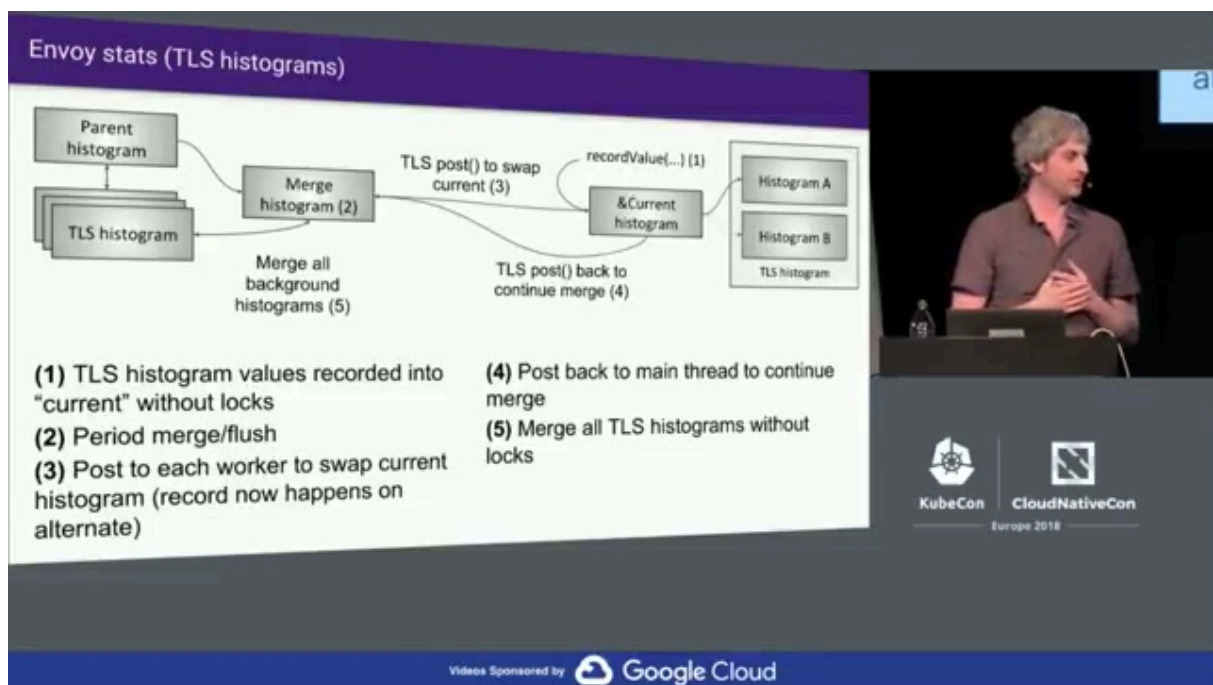
- a. **RCU와 TLS 활용:** 스레드 로컬 캐시를 사용하여 락 없이 통계 접근을 최적화한다.
- b. **데이터 저장 위치:**
- 카운터와 게이지는 **공유 메모리**에 저장된다.
 - 히스토그램은 **프로세스 메모리**에 저장된다. (재시작 시 데이터 손실이 운영상 큰 문제가 되지 않기 때문)

- c. **스레드 로컬 캐시**: 스레드가 통계를 요청할 때 먼저 캐시를 확인하고, 없으면 중앙 스토어에서 가져온 후 캐시에 추가한다.
- d. **메모리 누수 방지**: 리스너나 클러스터가 삭제될 때 메인 스레드가 워커 스레드들에게 캐시 데이터를 삭제하도록 알린다.

3. TLS 히스토그램

- a. 각 워커 스레드에 **TLS 히스토그램**을 두어 락 없이 값을 누적한다.
- b. 병합 시, 각 워커 스레드의 히스토그램을 **락 없이** 합친다.

4. 요약 및 향후 계획



1. Envoy의 핵심 가치

- a. **개발자 생산성 향상**: C++를 선택한 이유 중 하나이다.
- b. **고성능**: 높은 처리량과 낮은 지연 시간을 목표로 한다.
- c. **확장성**: 'embarrassingly parallel' 아키텍처를 통해 높은 하드웨어 스레드 수에 맞춰 확장된다.
- d. **컨테이너 환경 최적화**: RCU와 TLS를 활용하여 컨테이너 환경에 적합하게 설계되었다.
- e. **확장성**: 모든 계층에서 **확장성(extensibility)**을 제공한다.

2. 커뮤니티 및 채용 정보

- a. Envoy 커뮤니티 성장에 기여하고 있으며, 채용 중이다.
- b. 질의응답 시간을 통해 추가적인 질문에 답변할 예정이다.